Unit 3

Arrays: Introduction, Declaration and Initialization of Arrays, Storage of Array in Computer Memory, Accessing Elements of Arrays, Operations on Array Elements, Assigning Array to Another Array, Dynamic Change of Array Size, Sorting of Arrays, Search for Values in Arrays, Class Arrays, Two-dimensional Arrays, Arrays of Varying Lengths, Three-dimensional Arrays, Arrays as Vectors.

Inheritance: Introduction, Process of Inheritance, Types of Inheritances, Universal Super Class-Object Class, Inhibiting Inheritance of Class Using Final, Access Control and Inheritance, Multilevel Inheritance, Application of Keyword Super, Constructor Method and Inheritance, Method Overriding, Dynamic Method Dispatch, Abstract Classes.

Interfaces: Introduction, Declaration of Interface, Implementation of Interface, Multiple Interfaces, Nested Interfaces, Inheritance of Interfaces, Default Methods in Interfaces, Static Methods in Interface.

Introduction

- Inheritance is the backbone of object-oriented programming (OOP).
- It is the mechanism by which a class can acquire properties and methods of another class.
- Using inheritance, an already tested and debugged class program can be reused in some other application.

Inheritance

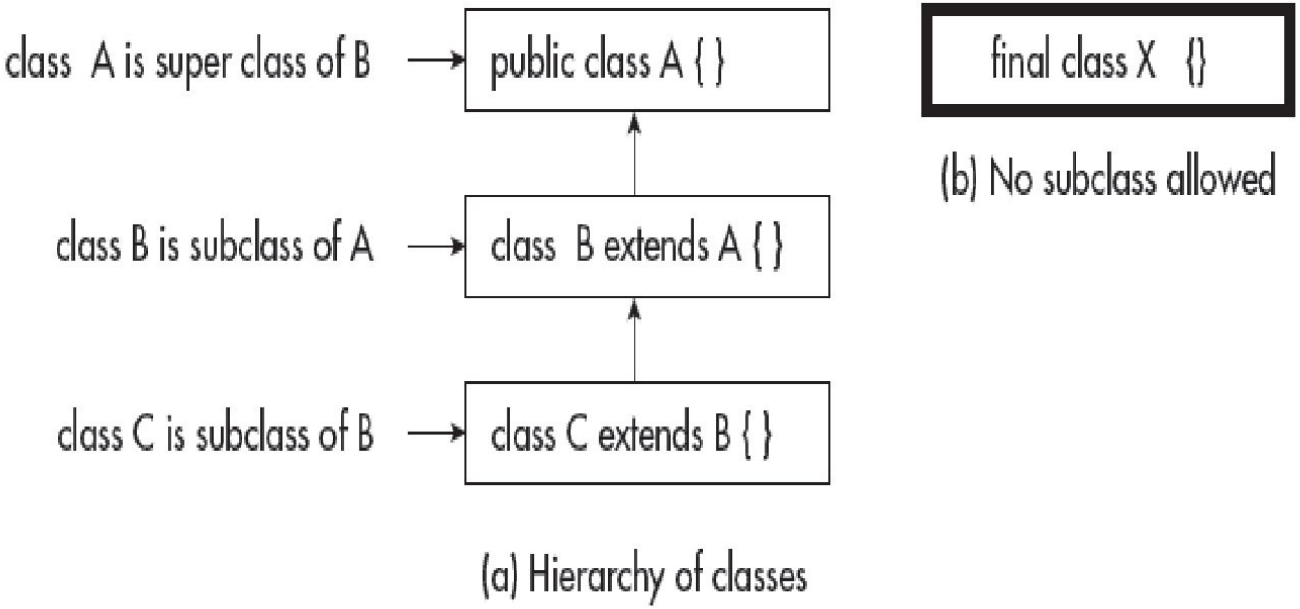
- Inheritance can be defined as the process where one class acquires the properties of another class
- The class which inherits the properties of other is known as subclass (derived class, child class) and the class whose properties are inherited is known as superclass (base class, parent class).
- Inheritance represents the **IS-A relationship**, also known as *parent-child* relationship.
- Use
    - For Method Overriding (so runtime polymorphism can be achieved).
    - For Code Reusability.

Syntax

```
class Subclass-name extends Superclass-name
{
  //methods and fields
}
```

- The extends keyword indicates that you are making a new class that derives from an existing class.

- **Super class** This is the existing class from which another class, that is, the subclass is generally derived.
  - In Java, several derived classes can have the same super class.
- **Subclass** A class that is derived from another class is called subclass.
  - In Java, **a subclass can have only one super class**



(a) Hierarchy of classes

(b) No subclass allowed

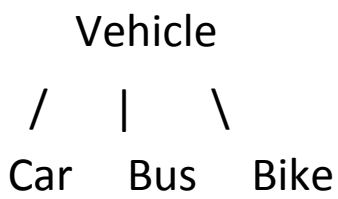
Benefits of Inheritance

- **Code Reusability**
  Already developed and tested code can be reused in another class without modifying the original code.
- **Easy Team Development**
  A large program can be divided into **classes and subclasses**. Different teams can work on different classes independently.
- **Easy Maintenance**
  If common functionality is placed in the superclass, changes made there can automatically benefit its subclasses.

Process of Inheritance

- Inheritance is a mechanism in Java by which a new class acquires the properties and methods of an existing class and can add its own specialized features.
- In Java, Inheritance means creating a new class from an existing class.
- The existing class is called the Superclass / Parent class. The new class is called the Subclass / Child class.
- The superclass contains common/general properties and methods.
- Sometimes, a particular group of objects needs additional or special characteristics. We have two choices:
  - Create a completely new class and write all the common characteristics again.
  - Extend the existing class and add only the special characteristics.

Suppose we have:



Vehicle contains common characteristics:
start()
stop()
speed()

Car may have additional characteristics:
airConditioner()

Bike may have:
kickStart()

So, instead of rewriting start(), stop(), and speed in every class, we **inherit them from Vehicle**.

Process of Inheritance

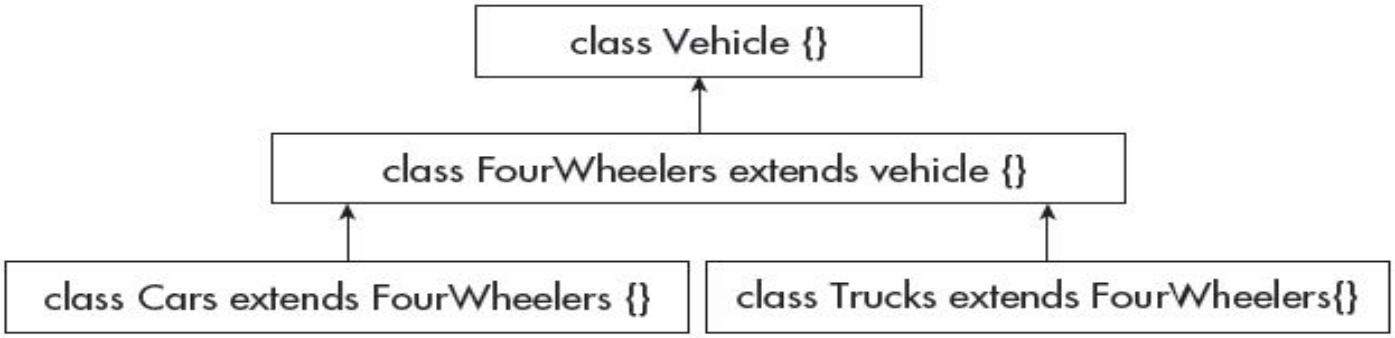
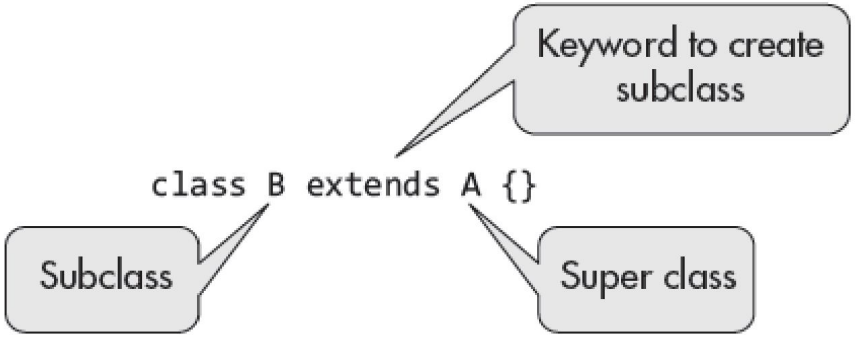


Fig. 8.4 Example for process of inheritance



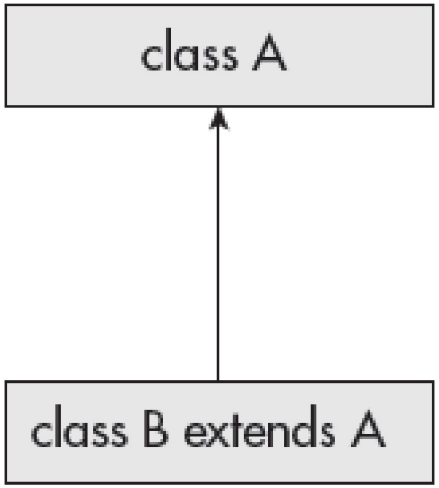
Types of Inheritance

The following types of inheritances are supported by Java.

1. Single inheritance
2. Multilevel inheritance
3. Hierarchical inheritance
4. Multiple inheritance using interfaces

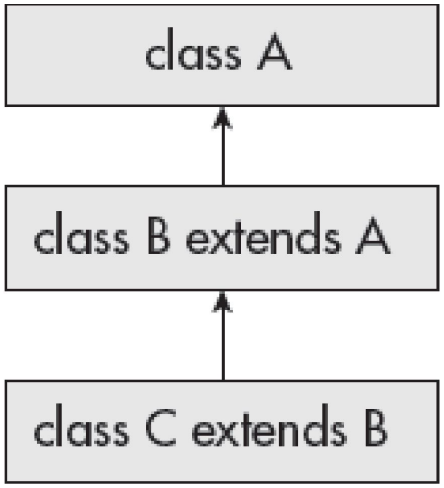
Types of Inheritance

(i) Single inheritance: It is the simple type of inheritance. In this, a class extends another one class only.



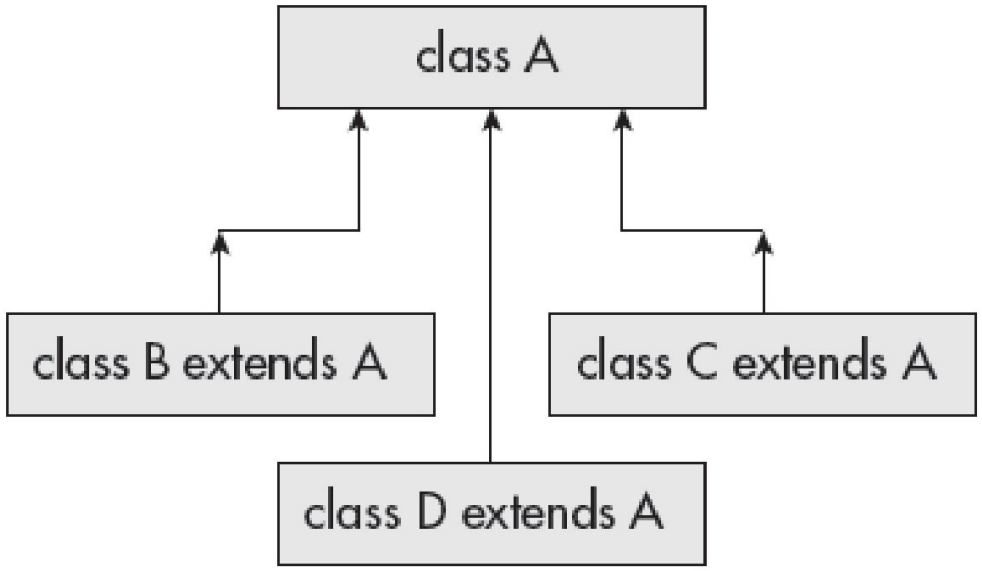
Types of Inheritances

(ii) Multilevel inheritance: When a class inherits from another class, and that derived class is further inherited by another class, it is called **Multilevel Inheritance**.



Types of Inheritances

- Hierarchical inheritance: When multiple subclasses inherit from a single superclass, it is called **Hierarchical Inheritance**.



(c)

Types of Inheritances

- Multiple inheritance: In this, a class is extending more than one class.
- Java does not support multiple inheritance.
- This implies that a class cannot extend more than one class.

Types of Inheritances

Table 8.1 Different types of inheritance

Inheritance types	*Application in program*

Single inheritance	`public class A{`
	`……..`
	`}`
	`public class B extends A{`
	`……..`
	`}`

Multilevel inheritance	`public class A{`
	`……..`
	`}`
	`public class B extends A{……..}`
	`public class C extends B{……..}`

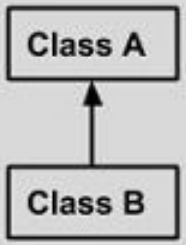
Hierarchical inheritance	`public class A{`
	`……..`
	`}`
	`public class B extends A{……..}`
	`public class C extends A{……..}`

Multiple inheritance using interfaces	`interface X`
	`{……}`
	`interface Y`
	`{……}`
	`public class A`
	`{……..}`
	`public class B extends A implements X, Y`
	`{……..}`

© Oxford University Press 2018. All rights reserved.

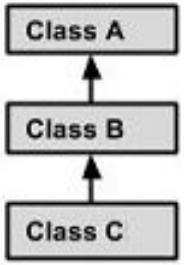
Types of Inheritances

Single Inheritance



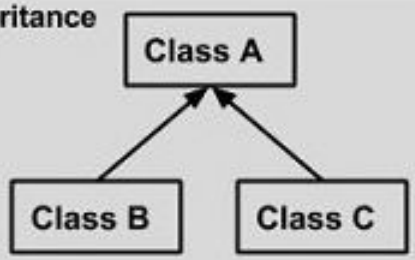

```
public class A {
   .....
}
public class B extends A {
   .....
}
```

Multi Level Inheritance



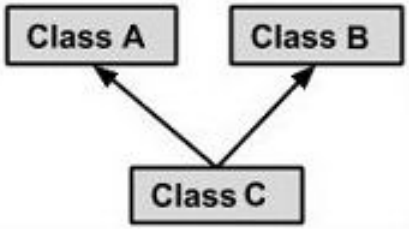

```
public class A { ..................}
public class B extends A {.....................}
public class C extends  B {..................... }
```

Hierarchical Inheritance




```
public class A { ..................}
public class B extends A {.....................}
public class C extends A {..................... }
```

Multiple Inheritance




```
public class A { ..................}
public class B {.....................}
public class C extends  A,B {
   .....................
} // Java does not support mutiple Inheritance
```

- A Super class reference variable can refer to a subclass object.
- Also called as upcasting.

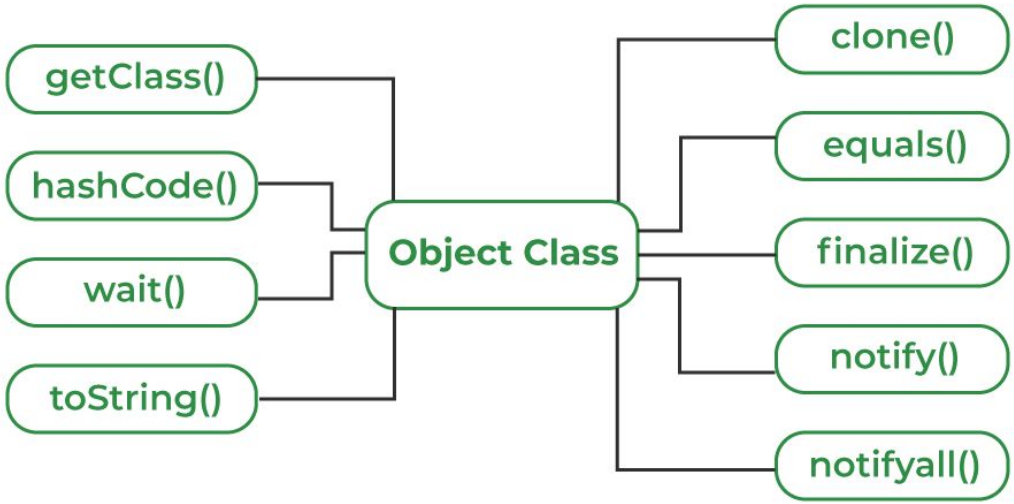
Universal Super Class—Object Class

- Object class is a special class and it is at the top of the class hierarchy tree.
- It is the parent class or super class of all in Java.
- Hence, it is called Universal super class.
- Object is at the root of the tree and every other class can be directly or indirectly derived from the Object class.

Universal Super Class—Object Class

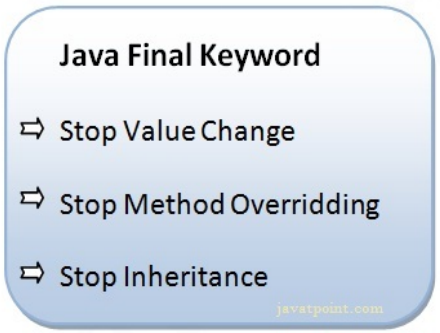
Method	Description
public String toString()	Returns the string representation of this object.
public int hashCode()	Returns the hashcode number for the given object.
public boolean equals(Object obj)	Compares the given object to this object.
Public final Class getClass()	Returns the Class class object of this object. The Class can be used to get the metadata of this class.

The default implementation of toString() returns a string that consists of the class name followed by the "@" character and the object's hash code in hexadecimal form



Inehibiting Inheritance of Class using Final

- To prevent modifications to the instance variable
- To Prevent method overriding
- To prevent inheritance



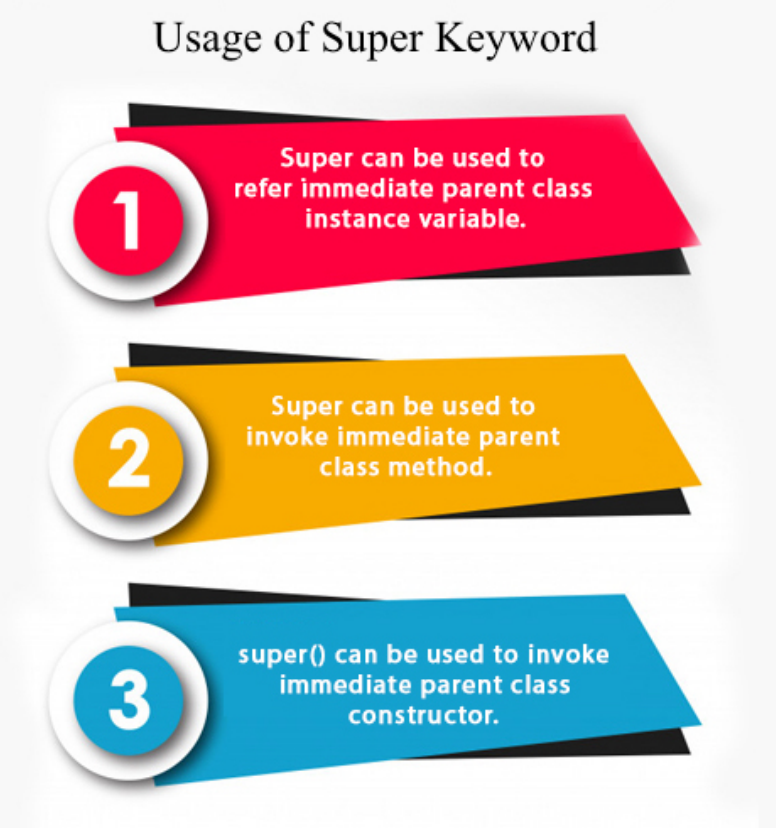
- Ex: FinalVariable.java, FinalMethod.java, FinalClass.java

Use of super

- Whenever the derived class inherits the base class features, there is a possibility that base class features are similar to derived class features and JVM gets an ambiguity.
- The super keyword in Java is a reference variable that is used to refer immediate parent class object.

- **Usage of Java super Keyword**
  - super is used to refer immediate parent class instance variable.
    - super.parent_instance_variable_name;
  - super is used to invoke immediate parent class method
    - super.parent_class_method_name();
  - super is used to invoke immediate parent class constructor.
    - super(arglist);
    - super(); //default

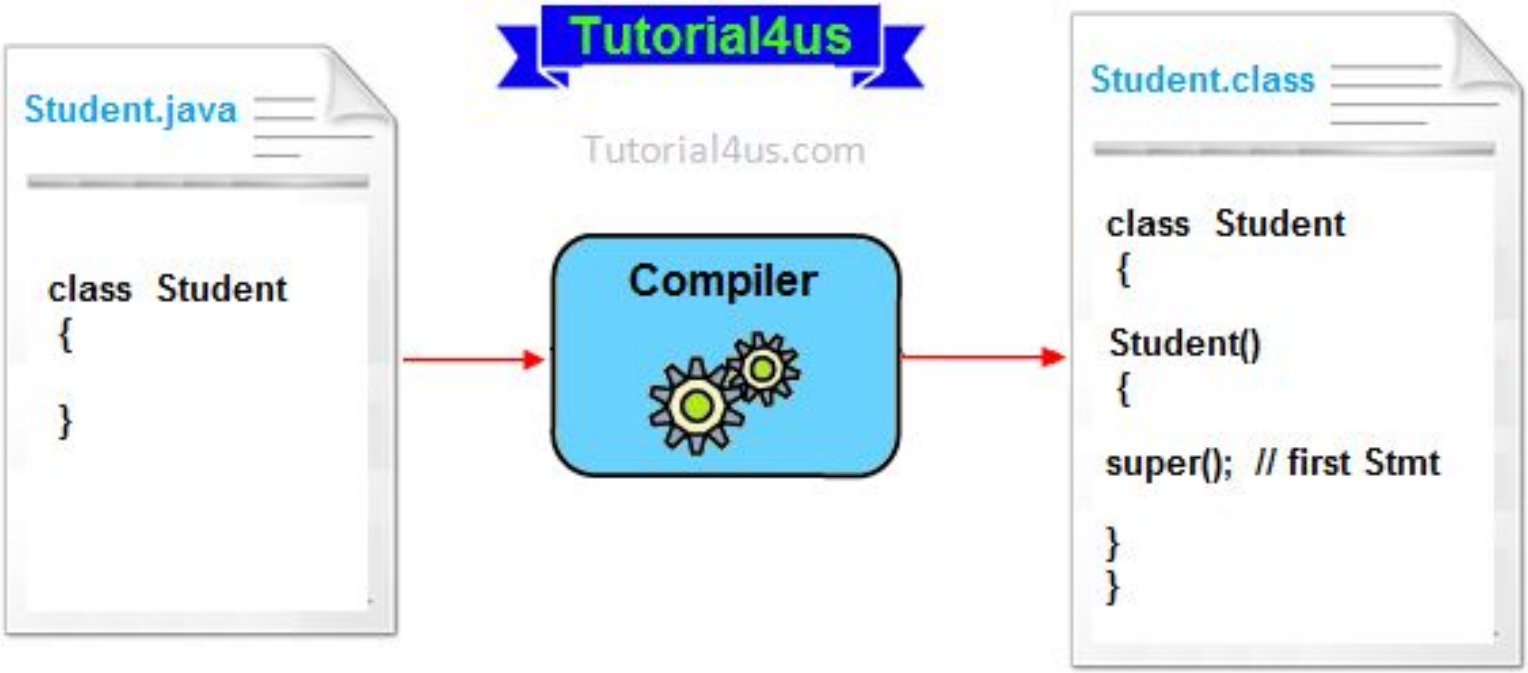
Super keyword is used to distinguish between the variable/methods having same name in super class and subclass.



Constructor and Inheritance

- Constructors of super class are not inherited by sub class but the constructor of the super class can be invoked from sub class using super().
- Constructor chaining takes place from bottom to top, while constructor execution takes place from top to bottom.
- To establish the connection between base class constructor and derived class constructors JDK provides two constructor statements:
  - super()
  - super(arglist)

.


Tutorial4us
Tutorial4us.com
Student.java
class Student
{
}
Compiler
Student.class
class Student
{
Student()
{
super(); // first Stmt
}
}

Super(...)

- **Super(...)** It is used for calling super class parameterize constructor from the context of derived class constructor.
- **Important rules**

  - Whenever we are using either super() or super(...) in the derived class constructors the **super** always must be as a first executable statement in the body of derived class constructor otherwise we get a compile time error.

Access Control and Inheritance

- When a **derived class (subclass)** inherits from a **superclass**, the access to superclass members depends on the **access specifier** used for those members.
- Java has **four access levels** actually:
  - public
  - protected
  - **default** (no keyword)
  - private
- The code for specifying access is

  access-specifier type member_name;

A public class can inherit all the public and protected members of super class. However, private members are not directly accessible to sub class. These are accessible with public methods of same class.

Table 8.3 Access specifiers

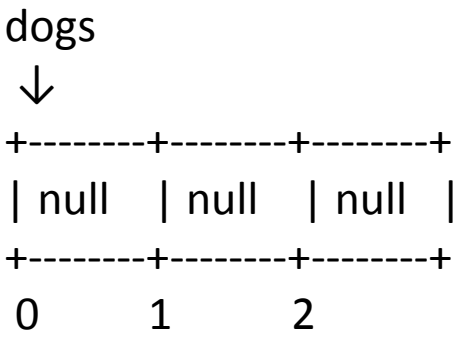
Access specifiers	*Access*
No access specifier	Access permitted to any other class belonging to the same package
public	Access permitted to any class in any package
protected	Access permitted to any subclass in any package, also to any class in the same package
private	Access permitted only to members of the same class. No outside code has access to a private member of a class

Table 8.4 Access control

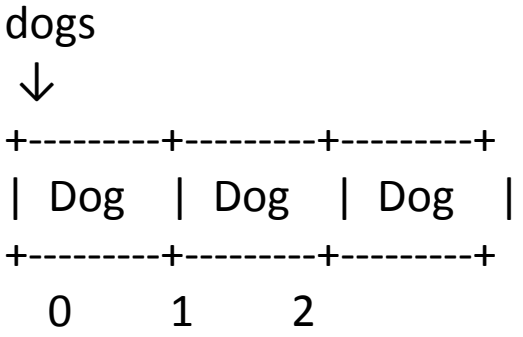
Class	*Access permitted (Yes/No)*			
	No specifier	*Public*	*Protected*	*Private*
Same class members	Yes	Yes	Yes	Yes
Subclass in same package	Yes	Yes	Yes	No
Any other class in same package	Yes	Yes	Yes	No
Subclass in different package	No	Yes	Yes	No
Any other class in different package	No	Yes	No	No

Array of Objects

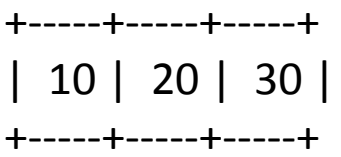
- An **array of objects** is an array whose elements are **references to objects**.
- Dog300[] dogs = new Dog300[3];



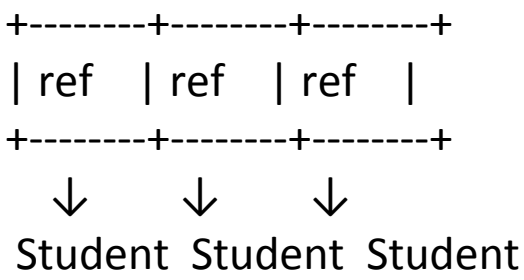
- dogs[0] = new Dog300();
- dogs[1] = new Dog300();
- dogs[2] = new Dog300();



- Primitive array
  - int[] a = new int[3];.



- Object array
  - Student[] s = new Student[3];

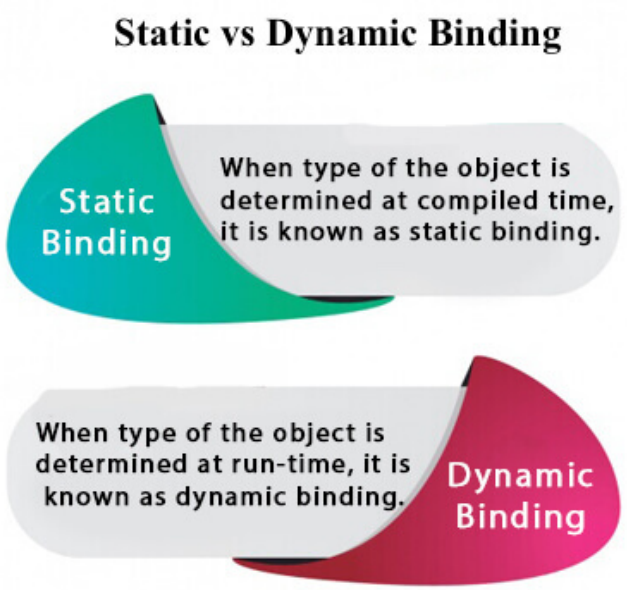


- An array of objects is an array that stores references to objects of a particular class.
- The objects themselves are created separately using new.

Method Overriding

- Method overriding is one of the ways in which polymorphism can be implemented.
- Polymorphism – objects take different forms at different environments.
- Method Overriding:
  - When both super class and sub class contain a method with same name and type signature - then the super class method is overriden by the sub class method.
  - When an overridden method is called from within its subclass, it will always refer to the version of that method defined by the subclass. The version of the method defined by the superclass will be hidden.

Types of Binding



Connecting a method call to the method body is known as binding.

There are two types of binding:

- Static binding / early binding /compile-time
- Dynamic binding /late /run-time

Static Binding (also known as Early Binding). –

- When type of the object is determined at compiled time(by the compiler), it is known as static binding.
- If there is any private, final or static method in a class, there is static binding.

- Dynamic Binding (also known as Late Binding).
  - type of the object is determined at run-time

Dynamic Method Dispatch

- Dynamic Method Dispatch is the mechanism by which runtime polymorphism is achieved for overridden methods in Java.
- It works on " A Super class reference variable refers to a sub class object".
- Java uses this fact to resolve calls to overridden methods at run time.
- Here is how?:
  - When an overridden method is called through a superclass reference, Java determines which version of the method will be executed based upon the type of the object being referred.

Abstract method

- An abstract method is a method without definition.
- Syntax:

```
abstract  return_type method_name(parameter_list);
```

Abstract class

- An abstract class is a class that contains one or more abstract methods.
- An abstract class can contain instance variables, constructors, concrete methods in addition to abstract methods.
- All the abstract methods of abstract class should be implemented in its sub classes.
- If any abstract method is not implemented in its subclasses, then that sub class must be declared as abstract.
- We can’t create an object to abstract class, but we can create references to abstract class.
- Also, you cannot declare abstract constructors or abstract static methods.

- QA?
  - Can you declare a class as abstract and final?

Interfaces Introduction

- **An interface is a blueprint/contract** that specifies what a class must do.
- An interface can contain: **Constants** (public static final variables) **Abstract methods** , **Default and static methods** (with Java 8+),

  Nested classes and interfaces.
- An interface **cannot have instance variables**.
- Its variables are implicitly public static final.
- An interface is **implemented by a class** using the implements keyword.
- Traditionally, **abstract methods in an interface do not have a body**; they end with a semicolon.
- The implementing class must provide the **method body** for the abstract methods.

```
interface Animal
{
   int legs = 4;            // Constant
   void sound();            // Abstract method
}
class Dog implements Animal
{
   public void sound()
   {
      System.out.println("Dog barks");
   }
}
class Test
{
   public static void main(String[] args)
   {
      Dog d = new Dog();
      d.sound();
   }
}
```

Introduction

- **A class that implements an interface must provide definitions for all abstract methods** declared in the interface, unless the implementing class itself is declared abstract.
- **An interface can be implemented by any number of unrelated classes.** Each class can provide its own implementation of the interface methods.
- **Different classes can provide different implementations of the same interface method.** The method signature (name and parameter list) must match the interface declaration, and the return type must be compatible.
- Therefore, **interfaces support polymorphism**. The same interface reference can refer to objects of different implementing classes, and the appropriate overridden method is executed at runtime.

```
interface Animal
{
   void sound();
}

class Dog implements Animal
{
   public void sound()
   {
      System.out.println("Dog barks");
   }
}

class Cat implements Animal
{
   public void sound()
   {
      System.out.println("Cat meows");
   }
}
```

```
class Test
{
   public static void main(String[]
args)
   {
      Animal a;

      a = new Dog();
      a.sound();

      a = new Cat();
      a.sound();
   }
}
```

Introduction

- **From Java SE 8 onwards, an interface can contain default and static methods with complete method definitions (method bodies).**
- A **default method is inherited by the implementing class**. The class can **override the default method** if it needs a different implementation.
- A class can **implement multiple interfaces**. This provides a way to achieve **multiple inheritance of behavior/contracts**. Java **does not allow a class to extend multiple classes**.
- For example: class C extends A, B // Not allowed

  class C implements A, B // Allowed

```
interface A
{
   default void show()
   {
      System.out.println("A's show()");
   }
}

interface B
{
   default void display()
   {
      System.out.println("B's display()");
   }
}
```

```
class C implements A, B
{
   // Can use both methods directly
}

class Test
{
   public static void main(String[] args)
   {
      C obj = new C();

      obj.show();
      obj.display();
   }
}
```

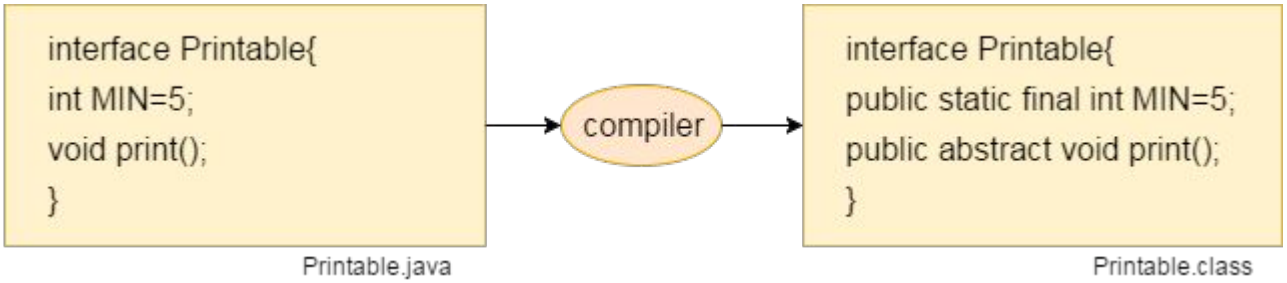
Similarities between Interface and Class

Feature	Class	Interface
Declaration	Declared using class keyword	Declared using interface keyword
Variables	Can contain instance, static and local variables	Cannot contain instance variables; variables are public static final constants
Methods	Can contain methods with complete definitions	Can contain abstract methods; default and static methods can have definitions
Object creation	Objects can be created using new	Objects cannot be created directly
Reference variable	Class reference variables are allowed	Interface reference variables are allowed

Inheritance	A class can extend only one class	A class can implement multiple interfaces
Multiple inheritance	Multiple inheritance of classes is not supported	Multiple interfaces can be implemented by a class
Access modifiers	Members can have private, protected, public, etc.	Abstract methods are implicitly public; interface constants are public static final
Constructors	Can have constructors	Cannot have constructors
Abstract methods	Can have abstract methods if the class is abstract	Can have abstract methods
Nested classes, interfaces	Can contain nested classes, interfaces	Can contain nested classes and interfaces
Implementation	A class can extend another class using extends	A class implements an interface using implements
Polymorphism	Supports runtime polymorphism through inheritance	Provides another mechanism for achieving runtime polymorphism

Dissimilarities between Class and Interface

- Interface cannot implement itself; it must be implemented by a class.
- An interface can contain only method headers followed by a semicolon.
- It cannot have the full definition of a method. The full definition is given in the class that implements it.
- The methods declared in the interface are implicitly public.
- An interface does not contain instance variables.
- The variables declared in an interface are implicitly public, static, and final,
- Interfaces cannot have a constructor like a class.
- An interface cannot extend a class nor can it have a subclass. It can only extend other interfaces.
- A class can extend (inherit) only one class but an interface can extend any number of interfaces.



Rules for Classes that Implement Interface

Abstract methods	A non-abstract class that implements an interface must provide concrete implementations of all abstract methods of that interface.
@Override annotation	The @Override annotation can be used on the implementing methods to indicate that they override/implement a method declared in the interface.
default methods	A default method has a complete definition in the interface and is inherited by the implementing class. The class may override it if required.
static methods	A static method has a complete definition in the interface, but it is NOT inherited by the implementing class. It is accessed using the interface name.

```
interface Animal
{
    void sound();       // Abstract method
    default void eat()
    {
        System.out.println("Animal eats");
    }
    static void info()
    {
        System.out.println("Animals are
living beings");
    }
}
class Dog implements Animal
{
    @Override
    public void sound()
    {
        System.out.println("Dog barks");
    }
}
```

```
class Test
{
    public static void main(String[] args)
    {
        Dog d = new Dog();

        d.sound(); // Implemented abstract method
        d.eat();        // Inherited default method

        Animal.info();   // Static method
    }
}
```

Types of Interfaces

Top level interfaces

- It is an interface that is not nested in any class or interface.

Nested interface

- It is an interface that is defined in the body of a class or interface.
- It is referred to by the outer interface or class and cannot be accessed directly.

```
class X{
    interface y{}
}
class A implements X.y { }
```

Generic interface

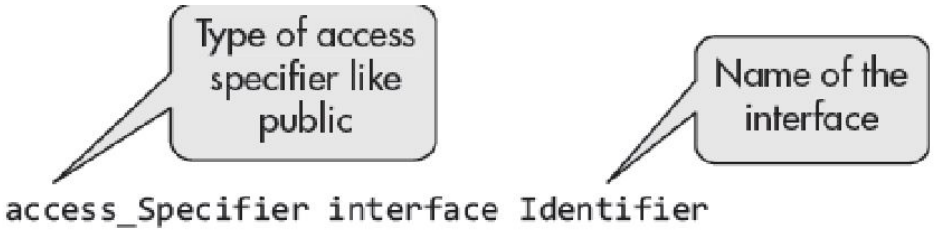
- We can develop more adaptable and reusable interfaces in Java since an interface can include a generic type parameter.
- This is implemented for different data types.

```
public interface MyInterface<T> {
   void doSomething(T value);
}
public class MyClass implements MyInterface<String> {
   public void doSomething(String value) {
      System.out.println("Doing something with " + value);
   }
}
public class AnotherClass implements MyInterface<int> {
   public void doSomething(int  value) {
      System.out.println("Doing something with " + value);
   }
}
```

Declaration of Interface

- Declaration of an interface starts with the access modifier followed by keyword interface.
- It is in then followed by its name or identifier that is followed by a block of statements;
    - These statements contain declarations of variables and abstract methods.
    - The variables defined in interfaces are implicitly public, static, and final.
    - They are initialized at the time of declaration.
    - The methods declared in an interface are public by default.
    - Since Java SE8, static and default methods with full definition can also be members of interface.

Declaration of Interface




```
{ //body
type variable1_name= value1;
 ----------------------------
type Method1_name (parameter list);

// Since Java SE8 an interface may have default methods.
default void display1 ()
{System.out.println("It is default method.");

//Since Java SE8 an interface may have static methods.
static void display2(){System.out.println("cosine 60 =" +
Math.cos(60*3.141/180));}
---------------------------- }
```

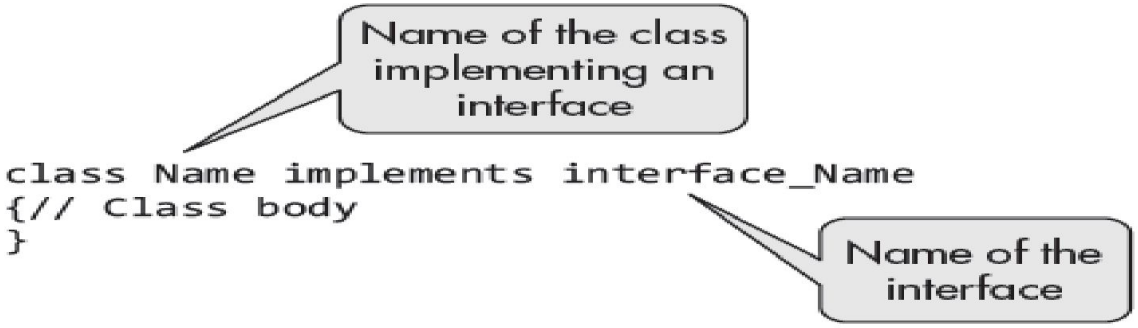
Implementation of Interface

Syntax:

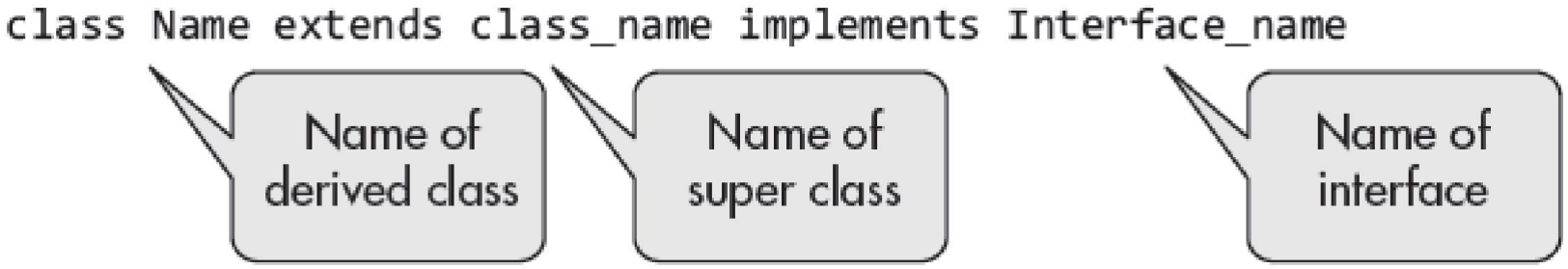
class *classname [extends superclass]*
[implements interface1 [,interface2...]] {
// class-body
}

Implementation of Interface

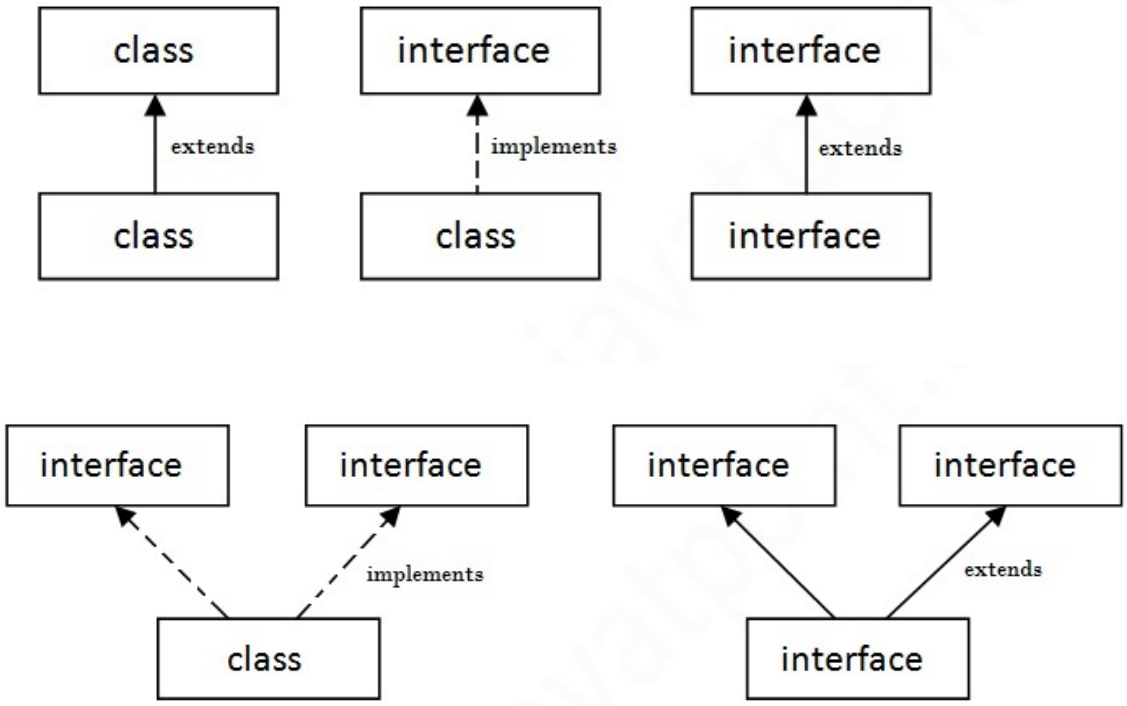
Declaration of class that implements an interface



If a class extends another class as well as implements interfaces, it is declared as



© Oxford University Press 2018. All rights reserved.

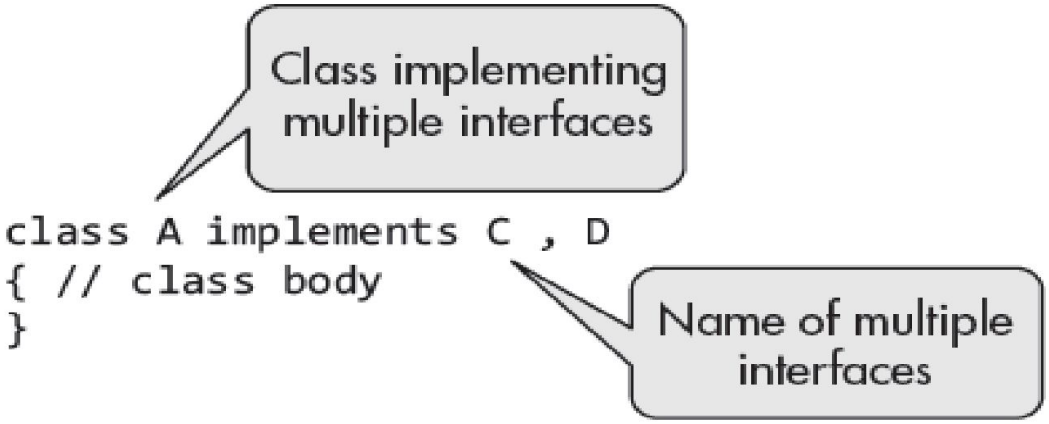


Multiple Inheritance in Java

Ex: FindAreas, use of constants

Multiple Interfaces

- Multiple interfaces can also be implemented in Java.
- For this, the class implements all the methods declared in all the interfaces.
- When the class is declared, names of all interfaces are listed after the keyword *implements* and separated by comma.
- As for example, if class A implements interfaces C and D, it is defined as



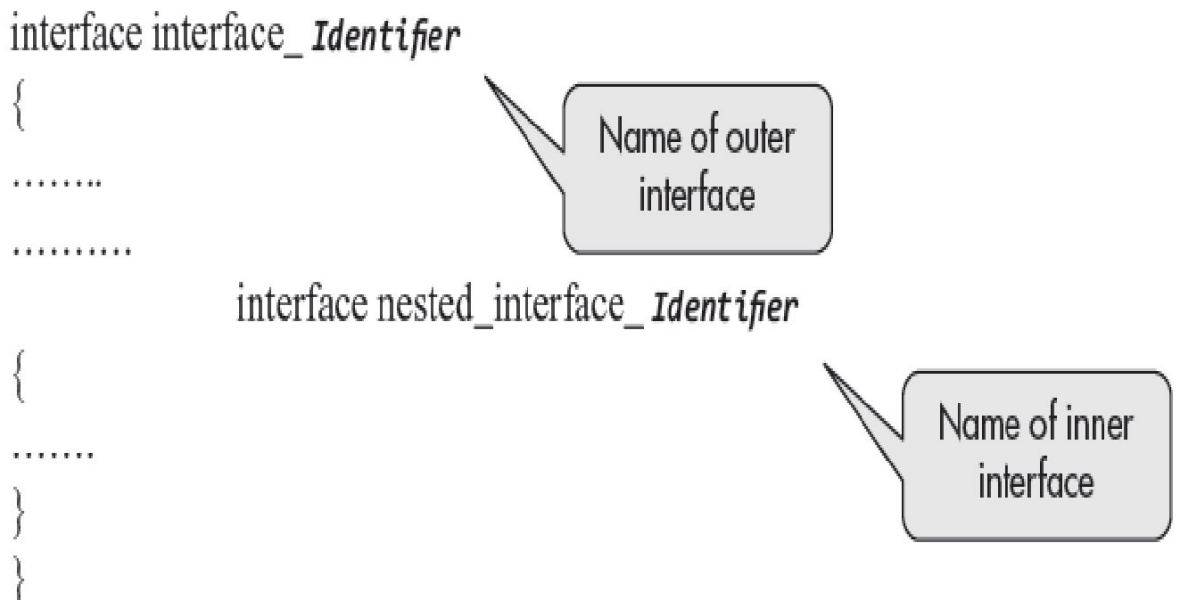
Interface References

- In Java, an interface cannot be instantiated.
- Therefore, it cannot be referenced directly.
- However, an object of class type, which extends the interface, can be used to assign a reference of that interface type.

- InterfaceReferences.java

Nested Interfaces

- An interface may be declared as a member of a class or in another
- interface.
- In nested concepts interfaces can only be declared as public or with default access.
- Syntax of nested interface in another interface is given as

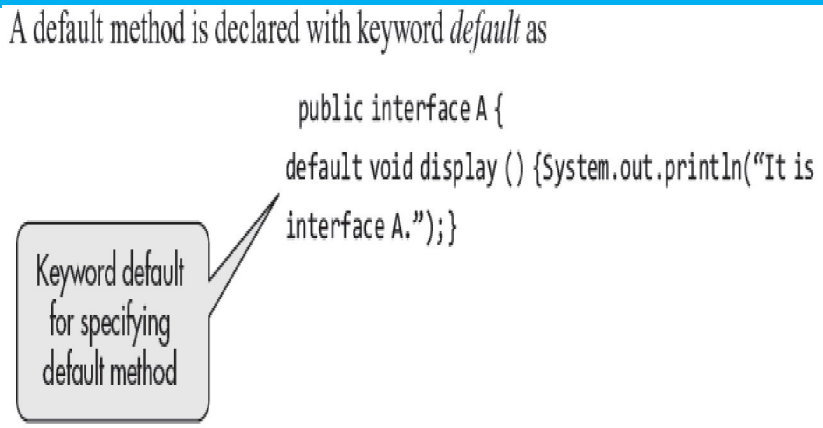


Inheritance of Interfaces

- One interface can inherit another by use of the keyword **extends**. The syntax is the same as like inheriting classes.
- When a class implements an interface that inherits another interface, it must provide implementations for all methods defined within the interface inheritance chain.

Default Methods in Interfaces

- Before Java 8, interfaces could have only abstract methods.
- The implementation of these methods has to be provided by sub class.



- To overcome this issue, Java 8 has introduced the concept of **default methods** which allow the interfaces to have methods with implementation details.
- Default methods are also known as defender methods or virtual extension methods.
- These methods can be overridden by implementing classes.

Rules

- If a class implements two or more interfaces, the interfaces cannot have default methods with the same signature because this would cause ambiguity as to which one to execute.
- In the case of methods with the same name, the compiler would choose by matching parameters.
- A default method cannot be declared final.
- If an interface A is extended by another interface B and both have a default method with same name, the definition of interface B will be chosen. Otherwise use super keyword.
  - Ex:DefaultMethodEx3.java

Static Methods in Interface

- The Java version 8 allows full definition of static methods in interfaces.
- The interfaces can have static methods which is similar to static method of classes.
- Can be referred with interface names.

  Interfacename.staticmethodname();
- NOTE: Static interface methods are not inherited by either an implementing class or subinterface
- These methods can not be overridden by classes or interfaces. Where as a default methods can be overridden.

© Oxford University Press 2018. All rights reserved.

Interface vs. Abstract Class

Abstract Class	Interface
Contains some abstract methods and some concrete methods	Only abstract methods, default and static methods
Contains instance variables	Only static and final variables
Doesn't support multiple inheritance	Supports
`public class Apple extends Food { … }`	`public class Person implements Student,Athlete,Chef { … }`

Use of toString()

- The toString() method returns the String representation of the object.
- If you print any object, Java compiler internally invokes the toString() method on the object.

  System.out.println(obj);
- So overriding the toString() method, returns the desired output, it can be the state of an object etc. depending on your implementation.